d_10 製造業モジュール間連携設計

対象: 製造業BPO 全16モジュール（①-⑧業務 + ⑨-⑬経営インテリジェンス + ⑭-⑯経営基盤）
上位文書: e_02b（既存8モジュール詳細設計）、e_02c（経営インテリジェンス5モジュール）、d_09（KPI設計）
実装場所: `workers/bpo/manufacturing/chain.py` （自動チェーン）、 `workers/bpo/manager/proactive_scanner.py` （能動提案）
Phase: Phase E-F（Week 6-9）
最終更新: 2026-03-28

---

0. 設計思想

なぜモジュール間連携が最重要か

```
中小製造業の経営判断は「部分最適の罠」にはまりやすい:
  ・見積は安く出して受注→粗利が低い→設備更新できない→品質低下→失注
  ・設備投資したが稼働率が上がらない→CF悪化→次の投資ができない
  ・材料相場が上がったが値上げ交渉しない→粗利率悪化→赤字転落

 → 16モジュールが「点」で動いても価値は半減
 → モジュール間でデータが流れ、連鎖的にアクションが起こる「面」の価値
 → 1+1=3 のシナジーが、月額¥55万のBPOを¥165万の価値に引き上げる
```

16モジュール一覧

#	モジュール	区分	Phase	月額価値
①	見積AI	業務	A	¥10-30万
②	生産計画AI	業務	A	¥5-10万
③	品質管理	業務	B	¥3-5万
④	在庫最適化	業務	B	¥3-5万
⑤	SOP管理	業務	C	¥2-3万
⑥	設備保全	業務	C	¥3-5万
⑦	仕入管理	業務	D	¥2-3万
⑧	ISO文書管理	業務	D	¥2-3万
⑨	PLインテリジェンス	経営	E-1	¥5-10万
⑩	設備投資シミュレーター	経営	E-1	¥3-8万
⑪	立地戦略AI	経営	E-2	¥2-5万
⑫	市場インテリジェンス	経営	E-2	¥3-5万
⑬	技術戦略AI	経営	E-3	¥3-5万
⑭	CF予測AI	経営基盤	F	¥5-8万
⑮	人材マネジメントAI	経営基盤	F	¥3-5万
⑯	事業承継AI	経営基盤	F	¥3-5万

---

1. 16x16 連携マトリクス（完全版）

1.1 マトリクス表記ルール

```
→  ：行→列 へデータを提供（一方向）
←  ：列→行 へデータを提供（一方向、→の逆方向記載）
↔  ：双方向のデータフロー
.  ：連携なし
```

1.2 完全マトリクス

	①見積	②生産	③品質	④在庫	⑤SOP	⑥設備	⑦仕入	⑧ISO	⑨PL	⑩設備投資	⑪立地	⑫市場	⑬技術	⑭CF	⑮人材	⑯承継
①見積	—	→A	→B	→C	.	→D	→E	.	→F	.	.	.	.	→G	→H	.
②生産	←A	—	→I	→J	→K	→L	→M	.	→N	→O	.	.	.	→P	→Q	.
③品質	←B	←I	—	.	→R	.	.	→S	→T	.	.	.	.	→U	.	.
④在庫	←C	←J	.	—	.	.	→V	.	→W	.	.	.	.	→X	.	.
⑤SOP	.	←K	←R	.	—	→Y	.	→Z	.	.	.	.	.	.	→AA	.
⑥設備	←D	←L	.	.	←Y	—	.	.	→AB	→AC	.	.	→AD	→AE	→AF	→AG
⑦仕入	←E	←M	.	←V	.	.	—	.	→AH	.	.	→AI	.	→AJ	.	.
⑧ISO	.	.	←S	.	←Z	.	.	—	.	.	→AK	.	.	.	→AL	.
⑨PL	←F	←N	←T	←W	.	←AB	←AH	.	—	→AM	.	→AN	→AO	→AP	.	→AQ
⑩設備投資	.	←O	.	.	.	←AC	.	.	←AM	—	↔AR	←AS	←AT	→AU	.	→AV
⑪立地	.	.	.	.	.	.	.	←AK	.	↔AR	—	←AW	.	.	→AX	.
⑫市場	.	.	.	.	.	.	←AI	.	←AN	→AS	→AW	—	→AY	.	.	.
⑬技術	←G	.	←U	.	.	←AD	.	.	←AO	→AT	.	←AY	—	.	→AZ	→BA
⑭CF	←H	←P	.	←X	.	←AE	←AJ	.	←AP	←AU	.	.	.	—	.	→BB
⑮人材	.	←Q	.	.	←AA	←AF	.	←AL	.	.	←AX	.	←AZ	.	—	→BC
⑯承継	.	.	.	.	.	←AG	.	.	←AQ	←AV	.	.	←BA	←BB	←BC	—

1.3 各連携の詳細（コード別）

業務モジュール間（①-⑧）

コード	方向	データフロー	内容
A	①→②	受注確定データ	見積承認→生産計画への受注投入（品名/数量/納期/工程）
B	①→③	見積時の品質要件	要求公差/表面粗さ/材質→品質検査基準の自動設定
C	①→④	材料所要量	見積確定→材料BOM展開→在庫引当/発注予約
D	①→⑥	設備稼働要求	受注→必要設備×必要時間→稼働計画への反映
E	①→⑦	材料発注トリガー	見積確定→BOM展開→仕入先への発注書ドラフト
G	①→⑬	引合い技術分析	引合い内容の技術要件→保有技術との適合性フィードバック
I	②→③	生産ロット情報	生産計画のロット→品質管理の検査ロット紐付け
J	②→④	生産消費予測	生産計画→材料消費タイミング→在庫引当スケジュール
K	②→⑤	作業指示	生産計画→該当工程のSOP自動表示（作業者端末）
L	②→⑥	設備使用計画	生産計画→設備別稼働スケジュール→保全タイミング調整
M	②→⑦	材料必要日	生産計画→材料の必要日→仕入先への納期指定
R	③→⑤	不良原因→SOP改訂	品質不良の原因分析→該当SOPの改訂提案
S	③→⑧	品質記録	品質データ（Cpk/不良率）→ISO文書の品質実績セクション
V	④→⑦	発注要求	安全在庫割れ→自動発注書生成
Y	⑤→⑥	設備操作手順	SOP更新→設備操作マニュアルとの整合性チェック
Z	⑤→⑧	手順書管理	SOP改訂履歴→ISO文書管理の文書改訂管理

業務→経営（①-⑧ → ⑨-⑬）

コード	方向	データフロー	内容
F	①→⑨	加工種別売上・原価	見積確定データ→加工種別PLの売上/原価の元データ
H	①→⑭	受注残	受注済み未出荷→売掛金回収予測のインプット
N	②→⑨	稼働率・生産実績	設備稼働率→PLの間接費按分根拠
O	②→⑩	稼働率	設備稼働率の実績→投資判断の根拠（キャパ不足/余剰判定）
P	②→⑭	生産計画→売上予測	生産計画の出荷予定→売上入金タイミング予測
Q	②→⑮	必要工数	生産計画の工数→必要人員算出
T	③→⑨	不良コスト	不良品の手直し/廃棄コスト→PLの品質コスト計上
U	③→⑬	品質実績	加工種別のCpk平均→技術力の定量評価
W	④→⑨	在庫コスト	在庫金額・回転率→PLの在庫保管コスト
X	④→⑭	在庫資産	在庫金額の変動→運転資本のCFインパクト
AB	⑥→⑨	保全コスト	修繕費実績→PLの設備関連コスト
AC	⑥→⑩	MTBF/MTTR	設備の故障頻度→更新判断の定量根拠
AD	⑥→⑬	設備年齢マップ	設備の残存耐用年数→技術シフトリスク評価
AE	⑥→⑭	修繕費予測	MTBFベースの故障予測→修繕費のCF予測
AF	⑥→⑮	設備操作資格	設備操作に必要な資格→保有者マッピング
AG	⑥→⑯	設備時価	簿価vs実勢価格→事業価値評価のインプット
AH	⑦→⑨	材料費実績	仕入実績→PLの材料費
AI	⑦→⑫	調達コスト実績	材料調達の単価推移→市場相場との比較
AJ	⑦→⑭	発注残	発注済み未納品→買掛金支払予測
AK	⑧→⑪	環境規制適合	ISO14001の環境マネジメント→立地の環境規制チェック
AL	⑧→⑮	資格管理	ISO内部監査員等の資格管理→人材スキルマトリクス

経営モジュール間（⑨-⑬）

コード	方向	データフロー	内容
AM	⑨→⑩	赤字加工種別	PLで赤字の加工種別→設備更新/廃止シミュレーション提案
AN	⑨→⑫	材料費比率	加工種別の材料費比率→材料相場変動のPLインパクト算出
AO	⑨→⑬	粗利率ランキング	加工種別の粗利率→技術投資優先度の判断材料
AR	⑩↔⑪	新設工場連携	⑩新設工場モード→⑪に候補地要求 / ⑪からコスト情報を返却
AS	⑫→⑩	市場成長機会	成長業界の検知→新規設備投資の提案
AW	⑫→⑪	成長産業集積地	業界トレンド→立地スコアリングの動的重み調整
AY	⑫→⑬	業界トレンド	市場の成長/縮小→技術ポジショニング評価のインプット
AT	⑬→⑩	投資提案	技術ロードマップの設備提案→設備投資シミュレーターで回収計算

経営基盤モジュール（⑭-⑯）

コード	方向	データフロー	内容
AP	⑨→⑭	月次PL	月次PLから営業CFを予測
AQ	⑨→⑯	EBITDA	PLデータからEBITDA算出→企業価値評価
AU	⑩→⑭	投資計画	設備投資計画→投資CFの予測
AV	⑩→⑯	設備時価評価	設備の簿価vs実勢価格→純資産評価
AX	⑪→⑮	地域労働市場	候補地の有効求人倍率/製造業従事者数→採用可能性評価
AA	⑤→⑮	技能マトリクス	SOPの技能要件→保有者とのギャップ分析
AZ	⑬→⑮	技能育成計画	技術ロードマップの必要スキル→育成計画の策定
BA	⑬→⑯	技術資産評価	技術ポートフォリオ→事業価値のうち無形資産評価
BB	⑭→⑯	CF予測	将来CFの予測→DCF法による企業価値算出
BC	⑮→⑯	後継者評価	後継者候補の技能レベル→承継リスク評価

2. 未設計連携3箇所の詳細設計

2a. ⑨PL → ⑫市場: 材料費比率と材料相場の突合（コードAN）

概要

⑨PLインテリジェンスが算出する「加工種別ごとの材料費比率」と、⑫市場インテリジェンスが収集する「材料相場の変動」を突合し、PLへの具体的なインパクト金額を算出する。

データフロー

```
⑨PLインテリジェンス                    ⑫市場インテリジェンス

┌──────────────────────┐                ┌──────────────────────┐
│ process_pl_results[] │                │ commodity_data       │
│  ・射出成型           │                │  ・ポリカーボネート   │
│    revenue: ¥620万    │                │    change: +8%       │
│    material_cost:     │                │  ・アルミ A5052      │
│      ¥279万(45%)      │──┐          ┌──│    change: +12%      │
│  ・CNC旋盤            │  │          │  │  ・SUS304            │
│    revenue: ¥1,500万  │  │          │  │    change: +3%       │
│    material_cost:     │  ▼          ▼  └──────────────────────┘
│      ¥375万(25%)      │  ┌──────────────┐
└──────────────────────┘  │ 突合エンジン  │
                          │ (AN連携)      │
                          └──────┬───────┘
                                 ▼
                     ┌──────────────────────┐
                     │ PLインパクトアラート  │
                     │ 「ポリカ+8%→射出成型の │
                     │   粗利率が3.6pt低下見込み。│
                     │   値上げ交渉を推奨」  │
                     └──────────────────────┘
```

連携ロジック

```
def calculate_material_impact(
    process_pl: dict,        # ⑨の出力: 加工種別PL
    commodity_changes: dict, # ⑫の出力: 材料相場変動
    material_mapping: dict,  # 加工種別→主要材料のマッピング
) -> list[dict]:
    """材料相場変動のPLインパクトを算出する。"""
    alerts = []

    for process_type, pl in process_pl.items():
        # 加工種別の主要材料を特定
        primary_materials = material_mapping.get(process_type, [])

        for material in primary_materials:
            if material not in commodity_changes:
                continue

            change_rate = commodity_changes[material]["change"]  # 例: 0.08 (+8%)
            material_ratio = pl["material_cost"] / pl["revenue"]  # 例: 0.45

            # PLインパクト = 材料費比率 × 相場変動率
            pl_impact_rate = material_ratio * change_rate  # 例: 0.45 × 0.08 = 0.036
            pl_impact_yen = pl["revenue"] * pl_impact_rate  # 例: 620万 × 3.6% = 22.3万

            # 粗利率への影響
            current_margin = pl["gross_margin"]  # 例: 12.1%
            new_margin = current_margin - (pl_impact_rate * 100)  # 例: 12.1% - 3.6% =
8.5%

            if abs(change_rate) >= ALERT_THRESHOLD:  # 5%超でアラート
                alerts.append({
                    "process_type": process_type,
                    "material": material,
                    "change_rate": change_rate,
                    "material_ratio": material_ratio,
                    "pl_impact_rate": pl_impact_rate,
                    "pl_impact_yen_monthly": int(pl_impact_yen),
                    "margin_before": current_margin,
                    "margin_after": new_margin,
                    "severity": "CRITICAL" if new_margin < 10 else "WARNING",
                    "action": _generate_action(process_type, material, change_rate,
new_margin),
                })

    return alerts

ALERT_THRESHOLD = 0.05  # 相場変動5%超でアラート

# 加工種別→主要材料マッピング
PROCESS_MATERIAL_MAPPING = {
```

```
    "射出成型":   ["ポリカーボネート", "ABS", "ナイロン", "POM"],
    "CNC旋盤":   ["SS400", "S45C", "SUS304", "A5052", "C3604"],
    "MCセンタ":  ["SS400", "S45C", "SUS304", "A5052", "A7075"],
    "板金":      ["SPCC", "SUS304", "A5052"],
    "プレス":    ["SPCC", "SPHC", "SUS304"],
    "表面処理":  ["ニッケル", "クロム", "亜鉛"],
    "溶接":      ["SS400", "SUS304", "溶接ワイヤ"],
    "研磨":      [],  # 材料比率が低いため対象外
    "ワイヤーカット": ["真鍮ワイヤ"],
}
```

トリガー条件

トリガー	条件	アクション
相場変動アラート	主要材料の相場変動が±5%超	PLインパクト算出→Slack通知
月次PL完了時	⑨月次PL完了イベント	当月の材料費実績と相場推移を突合
四半期レポート	⑫四半期レポート生成時	全加工種別×全材料のインパクトマトリクス

出力例

```
[WARNING] 材料相場変動アラート

■ ポリカーボネート +8%（LME 前月比）
  影響:
    ・射出成型の材料費比率 45% × 相場+8% = PLインパクト ▼3.6%
    ・月額影響: ▼¥22.3万（年換算 ▼¥268万）
    ・粗利率: 12.1% → 8.5%（🔴 赤字リスクゾーン）

  推奨アクション:
    1. 得意先への値上げ交渉（+5%で粗利率10.5%に回復）
    2. 代替樹脂の検討（ABS: 相場安定、物性互換可能な用途あり）
    3. ロット集約による材料仕入値引き交渉

■ SUS304 +3%（鉄鋼スクラップ連動）
  影響:
    ・CNC旋盤の材料費比率 25% × 相場+3% = PLインパクト ▼0.75%
    ・月額影響: ▼¥11.3万（年換算 ▼¥135万）
    ・粗利率: 28.2% → 27.5%（🟢 許容範囲内）

  推奨アクション:
    → 現時点では対応不要。5%超で再アラート
```

2b. ⑨PL → ⑬技術: 粗利ランクと技術投資優先度（コードAO）

概要

⑨PLの加工種別粗利率ランキングと、⑬技術戦略のポジショニングスコア（成長性）を掛け合わせ、技術投資の4象限アクションを導出する。

2x2マトリクス

```
                    技術成長性（⑬）
                    高い                低い

              ┌──────────────────┬──────────────────┐
粗利率        │ ★ STAR           │ 🐄 CASH COW      │
高い（⑨）     │ 投資加速         │ 現状維持+効率化  │
              │ → ⑩で追加投資    │ → 利益を他に再投 │
              │ シミュレーション │ 資する原資に     │
              ├──────────────────┼──────────────────┤
              │ ❓ QUESTION MARK │ 🐕 DOG           │
低い（⑨）     │ 選択と集中       │ 撤退/縮小検討    │
              │ → 改善余地あれば │ → 設備売却/外注  │
              │ 投資、なければ   │ 化を提案         │
              │ 撤退判断         │                  │
              └──────────────────┴──────────────────┘
```

連携ロジック

```
def classify_tech_investment(
    process_pl_results: list[dict],     # ⑨の出力: 加工種別PL
    positioning_scores: list[dict],     # ⑬の出力: 技術ポジショニングスコア
) -> list[dict]:
    """粗利率×技術成長性の4象限分類を行う。"""

    # 閾値
    MARGIN_THRESHOLD = 0.20       # 粗利率20%を高低の境界
    GROWTH_THRESHOLD = 50         # 成長性スコア50点を高低の境界

    classifications = []

    for pl in process_pl_results:
        process_type = pl["process_type"]
        gross_margin = pl["gross_margin"] / 100  # % → 小数

        # ⑬からポジショニングスコアを取得
        positioning = next(
            (p for p in positioning_scores if p["process_type"] == process_type),
            None
        )
        if not positioning:
            continue

        growth_score = positioning["growth_score"]  # 0-100

        # 4象限分類
        if gross_margin >= MARGIN_THRESHOLD and growth_score >= GROWTH_THRESHOLD:
            quadrant = "STAR"
            action = "投資加速"
            detail = (
                f"{process_type}は高粗利({gross_margin:.1%})+成長技術(スコア
{growth_score})。"
                f"⑩設備投資シミュレーターで追加設備の回収計算を推奨"
            )
            priority = 1

        elif gross_margin >= MARGIN_THRESHOLD and growth_score < GROWTH_THRESHOLD:
            quadrant = "CASH_COW"
            action = "現状維持+利益確保"
            detail = (
                f"{process_type}は高粗利({gross_margin:.1%})だが技術成長性は低い(スコア
{growth_score})。"
                f"現状維持で利益を確保し、STAR象限への再投資原資に"
            )
            priority = 3

        elif gross_margin < MARGIN_THRESHOLD and growth_score >= GROWTH_THRESHOLD:
            quadrant = "QUESTION_MARK"
```

```
            action = "選択と集中"
            detail = (
                f"{process_type}は低粗利({gross_margin:.1%})だが成長技術(スコア
{growth_score})。"
                f"チャージレート見直し/自動化投資で改善可能か⑩で試算。改善見込みなしなら撤退"
            )
            priority = 2

        else:
            quadrant = "DOG"
            action = "撤退/縮小検討"
            detail = (
                f"{process_type}は低粗利({gross_margin:.1%})+衰退技術(スコア
{growth_score})。"
                f"設備売却/外注化を検討。浮いたリソースをSTAR象限に集中"
            )
            priority = 4

        classifications.append({
            "process_type": process_type,
            "gross_margin": gross_margin,
            "growth_score": growth_score,
            "quadrant": quadrant,
            "action": action,
            "detail": detail,
            "priority": priority,
        })

    return sorted(classifications, key=lambda x: x["priority"])
```

出力例

```
■ 技術投資4象限分析

  ★ STAR（投資加速）
    ・5軸MC: 粗利率35% × 成長性82 → 航空機/EV部品市場が急成長。追加投資推奨
      → ⑩試算: 5軸MC追加（¥7,000万）回収3.5年。GO_CONDITIONAL

  🐄 CASH COW（利益確保）
    ・CNC旋盤: 粗利率28% × 成長性38 → 安定だが成長余地は限定的。現有設備で効率最大化
    ・表面処理: 粗利率38% × 成長性42 → 高収益。利益をSTAR投資に充当

  ❓ QUESTION MARK（選択と集中）
    ・板金: 粗利率18% × 成長性55 → 自動化（ファイバーレーザー）で粗利改善の余地あり
      → ⑩試算: レーザー加工機（¥4,000万）回収4.1年。GO_CONDITIONAL

  🐕 DOG（撤退検討）
    ・射出成型: 粗利率12% × 成長性25 → 電力コスト高+材料高騰+成長鈍化
      → 提案: 外注化して設備売却。浮いた¥2,000万を5軸MC投資に充当
```

2c. ⑪立地 → ⑫市場: 成長産業集積地との連動（コードAW）

概要

⑫市場インテリジェンスが検知する成長業界トレンド（例: 半導体+25%）を、⑪立地戦略AIのスコアリングに動的に反映する。成長産業が集積する地域のスコアをブーストし、衰退産業に依存する地域のスコアを減衰させる。

連携ロジック

```
def adjust_location_scores_by_market_trends(
    candidates: list[dict],           # ⑪の候補地リスト
    industry_trends: dict,            # ⑫の業界トレンド（5年予測）
    company_process_types: list[str], # 自社の加工種別
) -> list[dict]:
    """市場トレンドに基づいて立地スコアを動的に調整する。"""

    for candidate in candidates:
        industry_composition = candidate.get("industry_composition", {})
        boost_score = 0.0

        for industry, share in industry_composition.items():
            if industry in industry_trends:
                growth = industry_trends[industry]["growth"]
                avg_growth = (growth[0] + growth[1]) / 2  # 成長率の中央値

                # 成長産業の集積度が高い地域はブースト
                # 衰退産業の集積度が高い地域はペナルティ
                boost_score += share * avg_growth * MARKET_BOOST_WEIGHT

        # サプライチェーンスコア（20点満点）に加減算
        # 最大±5点の調整（スコアの25%を上限）
        adjustment = max(-5, min(5, boost_score * 100))
        candidate["supply_chain_score"] += adjustment
        candidate["total_score"] += adjustment
        candidate["market_trend_adjustment"] = {
            "adjustment_points": round(adjustment, 1),
            "reason": _generate_adjustment_reason(industry_composition,
industry_trends),
        }

    # スコア順で再ソート
    return sorted(candidates, key=lambda x: x["total_score"], reverse=True)

MARKET_BOOST_WEIGHT = 50  # 重み係数

# 業界→加工種別の親和性マッピング
INDUSTRY_PROCESS_AFFINITY = {
    "半導体製造装置": ["CNC旋盤", "MCセンタ", "5軸MC", "研磨", "ワイヤーカット"],
    "自動車_EV":     ["CNC旋盤", "MCセンタ", "プレス", "射出成型"],
    "航空宇宙":      ["5軸MC", "ワイヤーカット", "表面処理"],
    "医療機器":      ["CNC旋盤", "MCセンタ", "研磨"],
    "ロボット":      ["CNC旋盤", "MCセンタ", "板金"],
}
```

動的ブーストの例

```
■ 立地スコア動的調整の例

候補地A: 熊本テクノリサーチパーク
  基本スコア: 72点
  産業構成: 半導体40% / 自動車EV20% / 産業機械30% / その他10%
  成長性: 半導体+25% × 40% + EV+50% × 20% + 産機+5% × 30% = +12.5%
  調整: +5.0点（上限）
  最終スコア: 77点 ↑
  理由: 「TSMC/ソニー効果で半導体関連の受注機会が拡大中。
         御社のCNC旋盤+MC技術は半導体装置部品に適合」

候補地B: 浜松テクノポリス
  基本スコア: 78点
  産業構成: 自動車ICE50% / 自動車EV15% / 産業機械25% / その他10%
  成長性: ICE−25% × 50% + EV+50% × 15% + 産機+5% × 25% = −3.5%
  調整: −1.8点
  最終スコア: 76.2点 ↓
  理由: 「自動車ICE比率50%の地域。EV化で構造転換リスクあり。
         ただしEV部品需要は15%で成長中、転換対応次第では好立地」
```

---

3. 追加3モジュール（⑭⑮⑯）の連携設計

3.1 ⑭CF予測AI

パイプライン概要

```
実装: workers/bpo/manufacturing/pipelines/cashflow_forecast_pipeline.py
クラス: CashflowForecastPipeline
エントリ: run_cashflow_forecast_pipeline(company_id, input_data)
```

連携元と入力データ

連携元	コード	入力データ	用途
⑨PL	AP	月次PL（売上/原価/営業利益）	営業CFの予測ベース
⑩設備投資	AU	投資計画（金額/時期）	投資CFの予測
⑦仕入	AJ	発注残（未納品の発注書）	買掛金支払スケジュール
①見積	H	受注残（確定見積）	売掛金回収スケジュール
④在庫	X	在庫金額変動	運転資本のCFインパクト
⑥設備保全	AE	MTBFベース故障予測	突発修繕費の予測

CF予測ロジック

```
def forecast_cashflow(
    pl_data: dict,             # ⑨月次PL
    capex_plans: list[dict],   # ⑩投資計画
    purchase_orders: list[dict],# ⑦発注残
    sales_backlog: list[dict], # ①受注残
    inventory_delta: float,    # ④在庫変動額
    maintenance_forecast: dict,# ⑥修繕費予測
    forecast_months: int = 6,  # 予測期間
) -> dict:
    """6ヶ月先までのCF予測を生成する。"""

    monthly_forecasts = []

    for month in range(1, forecast_months + 1):
        # 営業CF（⑨PLベース）
        operating_cf = (
            pl_data["operating_profit"]
            + pl_data["depreciation"]         # 非現金費用の加算
            - _estimate_tax(pl_data)          # 税金支払
            - inventory_delta / forecast_months  # 運転資本変動
        )

        # 売掛金回収（①受注残ベース）
        ar_collection = sum(
            order["amount"] for order in sales_backlog
            if _collection_month(order) == month
        )

        # 買掛金支払（⑦発注残ベース）
        ap_payment = sum(
            po["amount"] for po in purchase_orders
            if _payment_month(po) == month
        )

        # 投資CF（⑩計画ベース）
        investing_cf = -sum(
            plan["amount"] for plan in capex_plans
            if plan["planned_month"] == month
        )

        # 突発修繕費（⑥予測ベース）
        maintenance_cf = -maintenance_forecast.get(f"month_{month}", 0)

        net_cf = operating_cf + ar_collection - ap_payment + investing_cf +
maintenance_cf

        monthly_forecasts.append({
            "month": month,
            "operating_cf": operating_cf,
```

```
            "ar_collection": ar_collection,
            "ap_payment": ap_payment,
            "investing_cf": investing_cf,
            "maintenance_cf": maintenance_cf,
            "net_cf": net_cf,
        })

    return {
        "forecasts": monthly_forecasts,
        "cumulative_cf": _cumulative(monthly_forecasts),
        "shortfall_month": _detect_shortfall(monthly_forecasts),  # 資金ショート月の検知
    }

# 売掛金回収条件（製造業の標準）
COLLECTION_TERMS = {
    "default": 60,       # 末締め翌月末（60日）
    "大手":    90,       # 末締め翌々月末（90日）
    "現金":    0,        # 即日
}

# 買掛金支払条件
PAYMENT_TERMS = {
    "default": 60,
    "手形":    120,      # 120日手形
}
```

⑭→⑯への出力

```
# CF予測 → 事業承継AI
cashflow_to_succession = {
    "annual_free_cashflow": annual_fcf,        # 年間FCF
    "fcf_growth_rate": fcf_cagr_3y,           # FCF成長率（3年CAGR）
    "cash_conversion_cycle": ccc_days,         # CCC（日数）
    "working_capital_ratio": wc_ratio,         # 運転資本比率
    "debt_repayment_capacity": dscr,           # DSCR（借入返済能力）
}
```

3.2 ⑮人材マネジメントAI

パイプライン概要

```
実装: workers/bpo/manufacturing/pipelines/hr_management_pipeline.py
クラス: HRManagementPipeline
エントリ: run_hr_management_pipeline(company_id, input_data)
```

連携元と入力データ

連携元	コード	入力データ	用途
②生産計画	Q	必要工数（設備×時間）	必要人員算出
⑤SOP	AA	技能マトリクス（工程×技能レベル）	技能保有者のギャップ分析
⑥設備保全	AF	設備操作資格一覧	資格保有者のマッピング
⑧ISO	AL	内部監査員等の資格	資格管理
⑪立地	AX	地域の労働市場データ	採用可能性評価
⑬技術	AZ	将来必要スキル	育成計画の策定

技能マトリクス連携

```
def build_skill_gap_analysis(
    sop_requirements: list[dict],    # ⑤SOPの技能要件
    equipment_certs: list[dict],     # ⑥設備操作資格
    staff_skills: list[dict],        # 現有人材のスキルDB
    production_plan: dict,           # ②生産計画
) -> dict:
    """技能ギャップ分析を行う。"""

    # 必要スキル一覧（⑤SOP + ⑥設備）
    required_skills = _merge_requirements(sop_requirements, equipment_certs)

    # 現有スキルとのギャップ
    gaps = []
    for skill in required_skills:
        holders = [s for s in staff_skills if skill["skill_id"] in s["skills"]]
        required_count = _calc_required_count(skill, production_plan)

        if len(holders) < required_count:
            gaps.append({
                "skill": skill["name"],
                "required": required_count,
                "current": len(holders),
                "gap": required_count - len(holders),
                "holders": [h["name"] for h in holders],
                "risk": "CRITICAL" if len(holders) <= 1 else "WARNING",
                "mitigation": _suggest_mitigation(skill, holders),
            })

    # 退職リスク（年齢×勤続年数ベース）
    retirement_risks = _assess_retirement_risk(staff_skills)

    return {
        "skill_gaps": gaps,
        "retirement_risks": retirement_risks,
        "succession_pairs": _suggest_succession_pairs(gaps, staff_skills),
    }

def _assess_retirement_risk(staff_skills: list[dict]) -> list[dict]:
    """退職リスクを評価する（年齢/勤続年数ベース）。"""
    risks = []
    for staff in staff_skills:
        age = staff.get("age", 0)
        tenure = staff.get("tenure_years", 0)
        unique_skills = staff.get("unique_skills", [])  # この人しかできない技能

        risk_score = 0
        if age >= 60: risk_score += 40
        elif age >= 55: risk_score += 25
        elif age >= 50: risk_score += 10
```

```
        if len(unique_skills) > 0: risk_score += 30  # 属人的スキル
        if tenure >= 30: risk_score += 15  # 長期勤続（退職時のインパクト大）

        if risk_score >= 50:
            risks.append({
                "staff_name": staff["name"],
                "age": age,
                "risk_score": risk_score,
                "severity": "CRITICAL" if risk_score >= 70 else "WARNING",
                "unique_skills": unique_skills,
                "succession_status": staff.get("successor", "未定"),
            })

    return sorted(risks, key=lambda x: x["risk_score"], reverse=True)
```

⑮→⑯への出力

```
# 人材データ → 事業承継AI
hr_to_succession = {
    "key_person_risks": retirement_risks,     # キーパーソンの退職リスク
    "successor_candidates": [                 # 後継者候補
        {
            "name": "田中太郎",
            "age": 35,
            "skills_coverage": 0.72,          # 社長の技能の72%をカバー
            "management_experience": "部門長3年",
            "gaps": ["見積判断", "銀行交渉", "顧客開拓"],
        }
    ],
    "skill_dependency_score": 0.85,           # 属人性スコア（1.0=完全属人的）
    "avg_successor_readiness": 0.45,          # 後継者準備度（0-1）
}
```

3.3 ⑯事業承継AI

パイプライン概要

```
実装: workers/bpo/manufacturing/pipelines/succession_planning_pipeline.py
クラス: SuccessionPlanningPipeline
エントリ: run_succession_planning_pipeline(company_id, input_data)
```

連携元と入力データ

連携元	コード	入力データ	用途
⑨PL	AQ	EBITDA（営業利益+減価償却費）	DCF法の元データ
⑩設備投資	AV	設備の簿価vs実勢価格	純資産法の資産評価
⑥設備保全	AG	設備時価（年齢/状態/市場価格）	有形資産評価
⑬技術	BA	技術ポートフォリオ評価	無形資産（技術力）の評価
⑭CF	BB	将来CF予測	DCF法による企業価値算出
⑮人材	BC	後継者候補の技能レベル	承継リスク評価

企業価値算出ロジック

```
def calculate_enterprise_value(
    pl_data: dict,               # ⑨EBITDA
    equipment_valuations: list,  # ⑩⑥設備の時価評価
    tech_portfolio: dict,        # ⑬技術資産評価
    cashflow_forecast: dict,     # ⑭将来CF予測
    hr_assessment: dict,         # ⑮人材評価
) -> dict:
    """3手法で企業価値を算出し、総合判断を提示する。"""

    # 手法1: 純資産法（時価ベース）
    tangible_assets = sum(e["market_value"] for e in equipment_valuations)
    intangible_premium = _calc_tech_premium(tech_portfolio)  # 技術力プレミアム
    net_asset_value = tangible_assets + intangible_premium - total_liabilities

    # 手法2: DCF法（⑭のCF予測を使用）
    annual_fcf = cashflow_forecast["annual_free_cashflow"]
    growth_rate = cashflow_forecast["fcf_growth_rate"]
    discount_rate = 0.08  # 中小製造業のWACC（リスクプレミアム込み）
    terminal_value = annual_fcf * (1 + growth_rate) / (discount_rate - growth_rate)
    dcf_value = sum(
        fcf / (1 + discount_rate) ** t
        for t, fcf in enumerate(cashflow_forecast["annual_fcfs"], 1)
    ) + terminal_value / (1 + discount_rate) ** len(cashflow_forecast["annual_fcfs"])

    # 手法3: EBITDA倍率法（中小製造業の相場: 3-6倍）
    ebitda = pl_data["ebitda"]
    ebitda_multiple = _determine_multiple(tech_portfolio, hr_assessment)  # 3-6倍
    ebitda_value = ebitda * ebitda_multiple

    # 承継リスク評価（⑮人材データ）
    succession_risk = _evaluate_succession_risk(hr_assessment)

    return {
        "net_asset_value": net_asset_value,
        "dcf_value": dcf_value,
        "ebitda_value": ebitda_value,
        "recommended_range": (
            min(net_asset_value, dcf_value, ebitda_value),
            max(net_asset_value, dcf_value, ebitda_value),
        ),
        "succession_risk": succession_risk,
        "action_plan": _generate_succession_action_plan(succession_risk, hr_assessment),
    }

# EBITDA倍率の決定要因
def _determine_multiple(tech_portfolio, hr_assessment):
    """技術力と人材の安定性でEBITDA倍率を調整する。"""
    base = 4.0  # 中小製造業の中央値
```

```
    # 技術力プレミアム
    if tech_portfolio.get("avg_cpk", 0) >= 1.50:
        base += 0.5  # 高精度加工
    if "5軸MC" in tech_portfolio.get("equipment_types", []):
        base += 0.5  # 高付加価値設備

    # 人材リスクディスカウント
    if hr_assessment.get("skill_dependency_score", 0) >= 0.80:
        base -= 1.0  # 属人性が高い
    if hr_assessment.get("avg_successor_readiness", 0) < 0.30:
        base -= 0.5  # 後継者が育っていない

    return max(3.0, min(6.0, base))
```

---

4. 自動チェーンルールの追加

4.1 追加チェーンルール一覧

以下を `workers/bpo/manufacturing/chain.py` に追加する。
パターンは `workers/bpo/sales/chain.py` の `trigger_next_pipeline` と同一構造。

ルール1: ⑨PL月次完了 → ⑭CF予測を自動実行

```
# ⑨PL月次完了 → ⑭CF予測
if pipeline_name == "manufacturing/pl_intelligence":
    if hasattr(result, "final_output") and result.final_output:
        period = result.final_output.get("period")
        asyncio.create_task(
            _run_cashflow_forecast(company_id, period)
        )
        triggered.append("manufacturing/cashflow_forecast")
```

トリガー条件: ⑨パイプラインが `completed` で終了
非同期: `asyncio.create_task` でブロックしない
失敗時: ログ記録のみ。⑨の結果に影響させない

ルール2: ⑨PLで粗利率急落 → ⑫市場で材料相場チェック → 値上げ提案自動生成

```
# ⑨PL粗利率急落 → ⑫市場チェック → 値上げ提案
if pipeline_name == "manufacturing/pl_intelligence":
    if hasattr(result, "final_output") and result.final_output:
        anomaly_alerts = result.final_output.get("anomaly_alerts", [])
        margin_alerts = [
            a for a in anomaly_alerts
            if a.get("type") == "margin_decline" and a.get("severity") in ("WARNING",
"CRITICAL")
        ]
        if margin_alerts:
            asyncio.create_task(
                _run_margin_decline_chain(company_id, margin_alerts)
            )
            triggered.append("manufacturing/margin_decline_chain")

async def _run_margin_decline_chain(company_id: str, margin_alerts: list) -> None:
    """粗利率急落チェーン: ⑫材料相場チェック → 値上げ提案生成。"""
    try:
        from workers.bpo.manufacturing.pipelines.market_intelligence_pipeline import (
            run_market_intelligence_pipeline,
        )
        # Step 1: 材料相場をチェック
        market_result = await run_market_intelligence_pipeline(
            company_id=company_id,
            input_data={
                "collection_type": "on_demand",
                "trigger": "margin_decline",
                "affected_processes": [a["process_type"] for a in margin_alerts],
            },
        )

        # Step 2: 材料相場が原因であれば値上げ提案を生成
        if market_result and market_result.material_impact:
            from workers.bpo.manufacturing.pipelines.pl_intelligence_pipeline import (
                generate_price_revision_proposal,
            )
            await generate_price_revision_proposal(
                company_id=company_id,
                margin_alerts=margin_alerts,
                material_impact=market_result.material_impact,
            )
            logger.info(f"chain: margin_decline_chain 完了 company_id={company_id}")

    except Exception as e:
        logger.error(f"chain: _run_margin_decline_chain 失敗（非致命的）: {e}")
```

ルール3: ⑮人材で退職リスクCRITICAL → ⑤SOPで技能伝承用手順書を自動生成

```
# ⑮退職リスクCRITICAL → ⑤SOP技能伝承手順書
if pipeline_name == "manufacturing/hr_management":
    if hasattr(result, "final_output") and result.final_output:
        critical_risks = [
            r for r in result.final_output.get("retirement_risks", [])
            if r.get("severity") == "CRITICAL" and r.get("unique_skills")
        ]
        if critical_risks:
            asyncio.create_task(
                _run_skill_transfer_chain(company_id, critical_risks)
            )
            triggered.append("manufacturing/skill_transfer_chain")

async def _run_skill_transfer_chain(company_id: str, critical_risks: list) -> None:
    """技能伝承チェーン: 退職リスク高の人材の固有スキル→SOP自動生成。"""
    try:
        from workers.bpo.manufacturing.pipelines.sop_management_pipeline import (
            run_sop_management_pipeline,
        )
        for risk in critical_risks:
            for skill in risk.get("unique_skills", []):
                await run_sop_management_pipeline(
                    company_id=company_id,
                    input_data={
                        "mode": "skill_transfer",
                        "skill_name": skill,
                        "current_holder": risk["staff_name"],
                        "priority": "urgent",
                        "reason": f"{risk['staff_name']}({risk['age']}歳)の退職リスク
CRITICAL",
                    },
                )
        logger.info(f"chain: skill_transfer_chain 完了 {len(critical_risks)}名分")
    except Exception as e:
        logger.error(f"chain: _run_skill_transfer_chain 失敗（非致命的）: {e}")
```

ルール4: ⑯事業承継でCRITICAL → ProactiveScannerで経営者面談を提案

```
# ⑯事業承継CRITICAL → ProactiveScanner経営者面談提案
if pipeline_name == "manufacturing/succession_planning":
    if hasattr(result, "final_output") and result.final_output:
        succession_risk = result.final_output.get("succession_risk", {})
        if succession_risk.get("severity") == "CRITICAL":
            asyncio.create_task(
                _propose_management_meeting(company_id, succession_risk)
            )
            triggered.append("manufacturing/succession_meeting_proposal")

async def _propose_management_meeting(company_id: str, succession_risk: dict) -> None:
    """ProactiveScannerに経営者面談を提案する。"""
    try:
        from db.supabase import get_service_client
        from datetime import datetime, timezone

        db = get_service_client()
        db.table("execution_logs").insert({
            "company_id": company_id,
            "pipeline": "manufacturing/succession_planning",
            "step": "proactive_proposal",
            "status": "pending",
            "payload": {
                "event": "succession_critical",
                "proposal_type": "management_meeting",
                "severity": "CRITICAL",
                "message": (
                    f"事業承継リスクがCRITICALです。"
                    f"理由: {succession_risk.get('reason', '後継者未定+社長年齢60歳超')}。"
                    f"経営者との面談を推奨します。"
                ),
                "suggested_agenda": [
                    "事業承継の現状認識の共有",
                    "企業価値算出結果の説明",
                    "後継者候補の技能ギャップ分析",
                    "3年間の承継ロードマップ策定",
                ],
                "available_for_proactive": True,
            },
            "executed_at": datetime.now(timezone.utc).isoformat(),
        }).execute()
        logger.info(f"chain: succession_meeting_proposal logged company_id=
{company_id}")
    except Exception as e:
        logger.warning(f"_propose_management_meeting failed: {e}")
```

4.2 チェーン実行フロー図

```
⑨PL月次完了
  ├─→ ⑭CF予測を自動実行（ルール1）
  └─→ 粗利率急落検知?
        └YES→ ⑫材料相場チェック → 値上げ提案自動生成（ルール2）

⑮人材評価完了
  └─→ 退職リスクCRITICAL?
        └YES→ ⑤SOP技能伝承手順書を自動生成（ルール3）

⑯事業承継評価完了
  └─→ 承継リスクCRITICAL?
        └YES→ ProactiveScanner経営者面談提案（ルール4）
```

---

5. ダッシュボードとの連動（d_09 KPI設計との接続）

5.1 7KPIのデータソースマッピング

KPI	データソース（主）	データソース（副）	連携コード
KPI-1: 月次売上	⑨PLインテリジェンス	①見積AI（受注確定額）	F
KPI-2: 粗利率	⑨PLインテリジェンス	⑫市場（材料相場→PL影響）	AN
KPI-3: 受注残月数	②生産計画AI	⑨PL（月商で割る）	N
KPI-4: 設備稼働率	②生産計画AI + ⑥設備保全	⑩設備投資（稼働率改善予測）	O, AC
KPI-5: 不良率(PPM)	③品質管理AI	⑬技術（品質力の定量評価）	U
KPI-6: 納期遵守率	②生産計画AI	④在庫（材料欠品による遅延）	J
KPI-7: キャッシュ残高	⑭CF予測AI	⑦仕入（買掛金） + ①見積（売掛金）	AP, AJ, H

5.2 アラート発火とモジュール間連携の対応

アラート	発火条件	トリガー元モジュール	連鎖するモジュール
C-1: 設備異常停止	設備ステータス `running` → `stopped`	⑥設備保全	→②生産計画（再スケジュール） →⑩設備投資（更新シミュレーション）
C-2: 重大品質クレーム	severity=critical	③品質管理	→⑤SOP（手順書見直し） →⑧ISO（是正処置記録）
C-3: キャッシュ枯渇予測	30日後キャッシュ<0	⑭CF予測	→⑦仕入（支払延期交渉リスト） →①見積（早期回収可能な売掛金リスト）
C-4: 大口顧客取引停止	月売上上位10%顧客が離脱	CRM/手動	→⑫市場（代替市場の探索） →②生産計画（稼働計画見直し）
C-5: 材料欠品ライン停止	安全在庫割れ+補充不可	④在庫最適化	→⑦仕入（緊急発注） →②生産計画（代替工程への振替）
W-1: 粗利率低下	前月比-5pt以上	⑨PL	→⑫市場（材料相場チェック） →⑬技術（4象限再評価）
W-2: 在庫回転率悪化	前月比-20%以上	④在庫最適化	→⑦仕入（発注量見直し） →⑨PL（在庫コスト影響額）
W-3: 設備稼働率低下	週次<60%	②生産計画	→⑩設備投資（設備統廃合検討） →⑮人材（余剰人員の配置転換）
W-6: 受注残枯渇	<1.0ヶ月	②生産計画	→⑫市場（新規開拓業界の提案） →⑬技術（新技術参入の検討）
W-7: 見積回答遅延	未回答3日超過	①見積AI	→⑮人材（見積担当の負荷分析）
W-8: MTBF超過設備	MTBFの80%到達	⑥設備保全	→⑩設備投資（更新シミュレーション自動起動） →⑭CF（修繕費予測更新）

5.3 ダッシュボード→モジュールへのドリルダウン

```
Level 0（7KPI）
  │
  ├─ KPI-2（粗利率）タップ → Level 1（加工種別PL比較）
  │     データソース: ⑨PLインテリジェンス
  │     追加情報: ⑫市場の材料相場影響額を併記
  │     アクション: 「値上げ提案を生成」→ ⑨→⑫連携チェーン起動
  │
  ├─ KPI-4（設備稼働率）タップ → Level 1（設備別稼働率）
  │     データソース: ②生産計画 + ⑥設備保全
  │     追加情報: ⑩設備投資の更新推奨フラグ
  │     アクション: 「更新シミュレーション」→ ⑥→⑩連携チェーン起動
  │
  ├─ KPI-7（キャッシュ残高）タップ → Level 1（CF予測6ヶ月）
  │     データソース: ⑭CF予測
  │     追加情報: ①受注残からの回収予定 + ⑦発注残からの支払予定
  │     アクション: 「資金繰り改善提案」→ ⑭が自動生成
  │
  └─ AIインサイト → 各モジュールの自動提案
        データソース: ⑨⑩⑫⑬⑭⑮⑯の各パイプライン出力
        表示: 優先度順（CRITICAL → WARNING → INFO）
```

---

6. 実装優先順位

Phase別の連携実装順

Phase	連携	実装内容	優先度
E-1	①-⑧→⑨	既存8モジュールからPLへのデータ連携	★最優先
E-1	⑥→⑩	MTBF超過→設備投資シミュレーション自動起動	★最優先
E-1	⑨→⑭	PL月次完了→CF予測自動実行（チェーンルール1）	★最優先
E-2	⑨↔⑫	材料費比率×材料相場突合（セクション2a）	高
E-2	⑫→⑪	成長産業集積地の動的ブースト（セクション2c）	中
E-3	⑨→⑬	粗利率×技術成長性の4象限分析（セクション2b）	高
F	⑭全連携	CF予測の全入力連携（6モジュールから）	高
F	⑮全連携	人材マネジメントの全入力連携（6モジュールから）	中
F	⑯全連携	事業承継の全入力連携（6モジュールから）	中
F	チェーン2	粗利急落→材料相場チェック→値上げ提案	高
F	チェーン3	退職リスクCRITICAL→SOP自動生成	中
F	チェーン4	承継CRITICAL→経営者面談提案	低

実装ファイル一覧

ファイル	内容
`workers/bpo/manufacturing/chain.py`	製造業BPO自動チェーンルール（新規）
`workers/bpo/manufacturing/pipelines/cashflow_forecast_pipeline.py`	⑭CF予測パイプライン（新規）
`workers/bpo/manufacturing/pipelines/hr_management_pipeline.py`	⑮人材マネジメントパイプライン（新規）
`workers/bpo/manufacturing/pipelines/succession_planning_pipeline.py`	⑯事業承継パイプライン（新規）
`workers/bpo/manufacturing/interconnect.py`	モジュール間連携ロジック（2a/2b/2c）（新規）

制約・前提条件

- Phase 1では自動チェーンのうちルール1（⑨→⑭）のみ実装。ルール2-4はPhase 2+
- ⑭⑮⑯のパイプラインはPhase Fで実装。Phase E完了後
- 16x16マトリクスの全連携を一度に実装するのではなく、Phase別に段階的に結線する
- 連携のデータ形式はPydanticモデルで厳密に定義し、モジュール間の契約とする
- 全連携はasync/awaitで実装。LangGraphの導入はPhase 2のHitLから
- チェーン失敗は上位パイプラインに伝播させない（`workers/bpo/sales/chain.py` と同じ原則）